COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/14/26	**DATE SUBMITTED:**	10/16/26
NAME:	Condino, Niel Vincent B. Guarin, Jan Mikaela C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12 Algorithmic Strategies

1. PROCEDURE OUTPUT

ILO A. Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Strategy	Algorithm	Analysis
Recursion	Binary Search	The recursive version of the binary search, where instead of just changing the values of l and r, the function is called within itself with different l or r values based on the satisfied condition
Brute Force	Linear Search	The brute force in linear search checks each element one by one, starting from index 0 until it finds the target or runs out of elements. It tries every possible way, with no shortcuts and no worst case.
Backtracking	Depth-First Search	DFS follows one path as far as it can. When a node has no unvisited neighbors, the algorithm goes back to the previous node and tries a different path. The returning step is backtracking since DFS does not restart the whole search when it reaches a dead end; instead, it explores the next option until the target is found or all nodes have been visited.
Greedy	Selection Sort	In every iteration, the algorithm selects the minimum value on the unsorted portion, then slots it in the last index of the said portion. It never rechecks this portion of the array, so there is no backtracking. Hence, it only looks “locally” for the best solution.
Divide-and-Conquer	Merge Sort	The algorithm tries to split the array into pairs(Divide); These pairs are merged and sorted until a sorted array is rebuilt(Conquer).

Table 12-1-1. Algorithmic Strategies and Examples

Memoization

Screenshot

```cpp
#include <iostream>
#include <algorithm>

int getMinSteps(int n,int memo[]){
        if ( n == 1 ){
            std::cout << "Base case reached\n";
            return 0;
        } // base case
        if( memo[n] != -1 ) {
            std::cout << "Already solved it, ";
            std::cout << "Solution for " << n << " is " << memo[n] << '\n';
            return memo[n];
        } // we have solved it already :)
        int r = 1 + getMinSteps(n - 1,memo); // '-1' step . 'r' will contain the optimal answer finally
        if( n%2 == 0 ) r = std::min( r , 1 + getMinSteps( n / 2,memo  )) ; // '/2' step
        if( n%3 == 0 ) r = std::min( r , 1 + getMinSteps( n / 3,memo )) ; // '/3' step
        memo[n] = r ; // save the result. If you forget this step, then its same as plain recursion.
        return r;
}

int main(){
    const int n = 10;
    int memo[n+1];
    //populate
    for (int i=1;i<=n;i++){
        memo[i] = -1;
    }
    std::cout << getMinSteps(n,memo) << '\n';
    //added shit
    std::cout << "Memo values {";
    for (int i=1;i<=n;i++){
        std::cout << memo[i]<<",";
    }

    std::cout << "}\n";

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> & C:\Users\TIPQC\Documents\HOA12\ILOA12.1.exe
Base case reached
Base case reached
Base case reached
Already solved it, Solution for 2 is 1
Already solved it, Solution for 3 is 1
Already solved it, Solution for 2 is 1
Already solved it, Solution for 4 is 2
Already solved it, Solution for 3 is 1
Already solved it, Solution for 5 is 3
3
Memo values {-1,1,1,2,3,2,3,3,2,3,}
```

Analysis

This uses the array memo with size n+1 to store all the already computed values. Each index is used as a key for an input, and the value is the already computed number from previous invocations. If the value is in the memo array, we just look it up instead of recomputing it, which saves runtime. This approach reduces the runtime from $O(3^n)$ *(exponential time)* to just $O(n)$*(linear time)*.

Table 12-2. Memoization Implementation

Bottom-Up DP

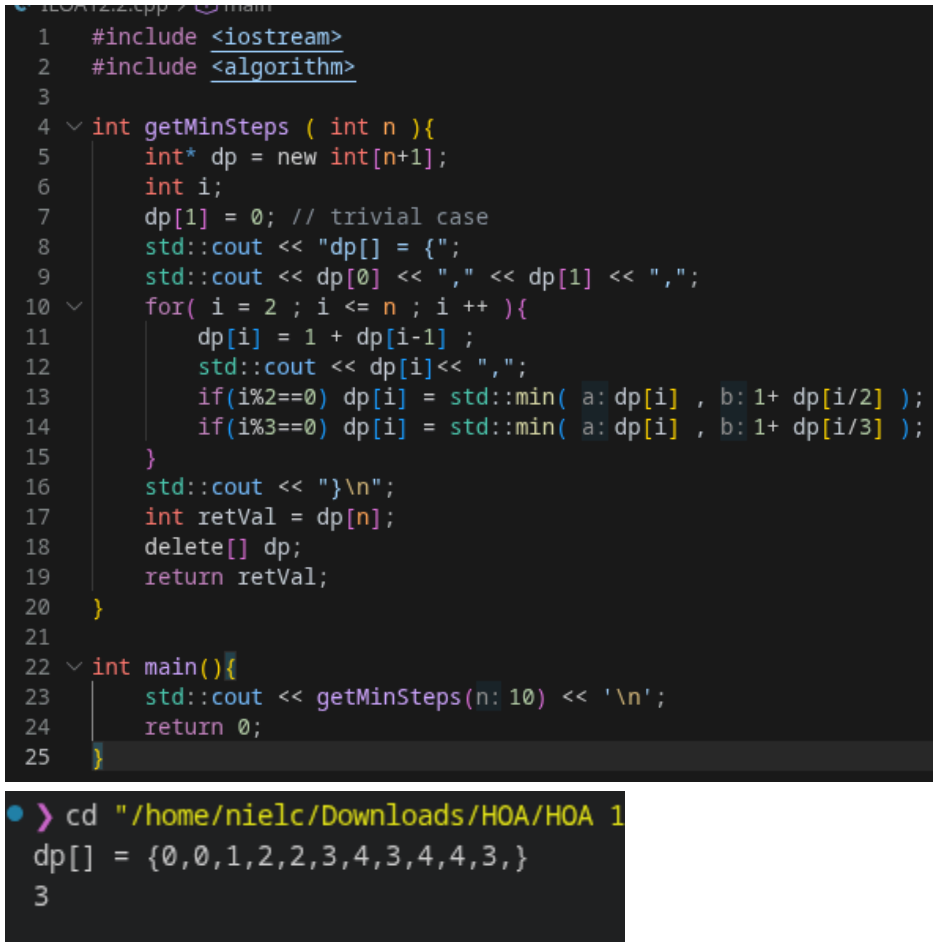
Screenshot	
Analysis	This code demonstrates how bottom-up dynamic programming solves the problem without recursion. The code creates a **dp** array where **dp [i]** stores the fewest steps needed to turn **i into 1,** and starts with dp [1] = 0. Next, a loop goes up from 2 to n, and for each number it assumes it will subtract 1 step, then checks if the number is divisible by 2 or 3, and it keeps the smaller result. Since the smaller result answers are already stored before they are needed, the final answer is in dp [n]. When the program runs **getMinSteps,** it prints 3, because 10 -> 9 -> 3 -> 1 takes three steps.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PSEUDOCODE

```
START
   Read M, N, K
   Read the matrix A

   Initialize memoization table to  -1

   FUNCTION findPaths(i, j, remaining)
      IF i or j is outside the matrix
         RETURN 0

      IF A[i][j] > remaining
         RETURN 0

      remaining = remaining - A[i][j]

      IF i = M-1 AND j = N-1
         IF remaining = 0
            RETURN 1
         ELSE
            RETURN 0

      IF result is already stored in memo[i][j][remaining]
         RETURN stored result

      right = fintPaths(i, j+1, remaining)
      down = findPaths(i+1, j, remaining)

      memo[i][j][remaining] = right + down

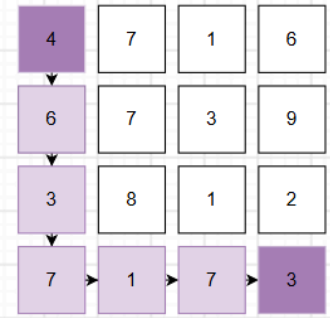
      RETURN memo[i][j][remaining]
   END FUNCTION

   answer = findPaths(0, 0, K)
   Display answer
END
```

Hand solution

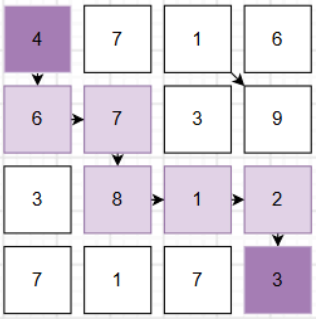
4 (0,0)	7 (0,1)	1 (0,2)	6 (0,3)
6 (1,0)	7 (1,1)	3 (1,2)	9 (1,3)
3 (2,0)	8 (2,1)	1 (2,2)	2 (2,3)
7 (3,0)	1 (3,1)	7 (3,2)	3 (3,3)

- Start at `(0,0)` with budget `31`.
- Subtract the current square's cost from the remaining budget.
- Move DOWN to `(row + 1, column)`.
- Move RIGHT to `(row, column + 1)`.
- If outside the array or the next square's cost exceeds the remaining budget, return `0`.
- If the destination `(3,3)` is reached with remaining budget `0`, return `1`; otherwise, return `0`.
- Add the results of the DOWN and RIGHT branches.



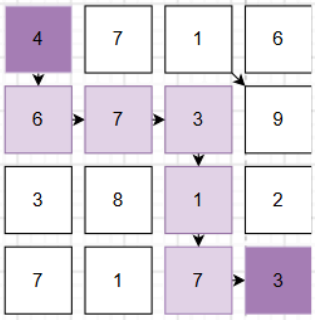
PATH 1

Path: 4 - 6 - 3 - 7 - 1 - 7 - 3



PATH 2

Path: 4 - 6 - 7 - 8 - 1 - 2 - 3



PATH 3

Path: 4 - 6 - 7 - 3 - 1 - 7 - 3

Code

```cpp
#include <cstring>
#include <iostream>

const int rowSize = 4;
const int columnSize = 4;

const int cost = 31;

int memo[rowSize][columnSize][cost +1] = {[0][0][0]=0};

int matrix[rowSize][columnSize] = {
    [0]={[0]=4,[1]=7,[2]=1,[3]=6},
    [1]={[0]=6,[1]=7,[2]=3,[3]=9},
    [2]={[0]=3,[1]=8,[2]=1,[3]=2},
    [3]={[0]=7,[1]=1,[2]=7,[3]=3},
};

int findPaths(int yCoord, int xCoord, int remaining){
    //Out of bounds check
    if (yCoord == rowSize || xCoord == columnSize){
        return 0;
    }

    int currentCost = matrix[yCoord][xCoord];

    //Insufficient currency check
    if (currentCost > remaining){
        return 0;
    }

    remaining -= currentCost;

    //Reached the bottom-right destination
    if (yCoord == rowSize - 1 && xCoord == columnSize - 1){
        if (remaining== 0){
            return 1;
        }
        else return 0;
    }

    //Memoization lookup
    if (memo[yCoord][xCoord][remaining] != -1){
        return memo[yCoord][xCoord][remaining];
    }

    int downTraverse = findPaths(yCoord: yCoord + 1, xCoord, remaining);
    int rightTraverse = findPaths(yCoord, xCoord: xCoord + 1, remaining);
```

```
48
49      //Save result to memo table and return
50      memo[yCoord][xCoord][remaining] = downTraverse + rightTraverse;
51
52      return memo[yCoord][xCoord][remaining];
53  }
54
55  int main(){
56      memset(s: memo, c: -1, n: sizeof(memo));//Initialize memo to -1
57      std::cout << "With a "<< cost <<" cost, There are "<< findPaths(yCoord: 0, xCoord: 0, remaining: cost) << " paths \n";
58      return 0;
59  }
```

Analysis

Condino:
To solve the problem given, a Recursive Depth-First Search with memoization is implemented. To find the path to the bottom-rightmost part of the array, it uses a DFS algorithm while keeping the remaining currency as a factor. If the currency is exhausted, the algorithm backtracks to the last position in the matrix, then finds a new path. To make sure it does not calculate the path for a given index, memoization is implemented. This array stores the path it calculated. It uses the position in the matrix and the remaining cost as an index in the memoization matrix. This reduces the runtime complexity from exponential (due to recalculations) to around cubic runtime(row x column x cost). This technique, however, hurts space complexity because of the space the memoization matrix uses.

Guarin:
To count the paths that cost exactly 31, the code uses recursive depth - first search that moves only down or right from the top - left cell. It carries the remaining currency along as a parameter. When a cell costs more than what is left, or the position goes past the edge of the matrix, the branch stops and returns to the previous cell so the search can try the other direction. When the search reaches the bottom - right cell, it counts one path only if the currency has dropped to exactly zero. To avoid repeating work, each result is saved in a three - dimensional memo array indexed by row, column, and remaining currency, so a state that comes up again is answered from the table instead of being recomputed. Without this, the number of recursive calls grows exponentially with the size of the matrix, but with memoization it drops to roughly rows x columns x cost.

Screenshots of demonstration

```
❯ cd "/home/nielc/Downloads/HOA/HOA 12/" &&
With a 31 cost, There are 3 paths
```

```
❯ cd "/home/nielc/Downloads/HOA/HOA
With a 25 cost, There are 0 paths
```

```
❯ cd "/home/nielc/Downloads/HOA/HOA 12/
With a 24 cost, There are 0 paths
```

Conclusion	
Condino	This activity taught me about the different algorithmic strategies such as recursion, brute force, backtracking, etc. Each of these strategies has its own strengths and weaknesses. Knowing what to apply in a certain problem is crucial to satisfy or sometimes optimally solve a problem. With the past activities, I already applied some algorithmic strategies such as recursion, backtracking, and divide and conquer. Not only in this course but also in my previous programming courses, I now have a deeper understanding of how they work. The supplementary material provided a challenge, and it is fun to think of a solution that can be implemented. Overall, I will remember these strategies and try to implement them in my future programming projects.
Guarin	In this activity, I learned that recursion, brute force, backtracking, greedy, and divide-and-conquer are different ways of solving a problem, and that dynamic programming works best when a problem has overlapping subproblems. At first, this activity confused me, especially the memoization part and what to store in the memo table, but with the help of my groupmate, I was able to understand how it works. I also found out that greedy doesn't always give the best answer, like when n = 10 in Minimum Steps to One, where greedy took 4 steps but the best answer was only 3. We tried both memoization and bottom-up DP for that problem and got the same results, but bottom - up felt safer because it has no recursion and no risk of stack overflow. I still need to practice writing the recurrence before coding and converting my solutions to bottom - up.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.